

SIMATS ENGINEERING
Department of Computer Science & Engineering
ASSESSMENT TOOL 1- ANALYTICAL PROBLEM SOLVING

Course Code:	CSA12	Course Title:	Computer Architecture
CO Assessed:	CO2	Assessment:	ASSESSMENT TOOL 1- ANALYTICAL PROBLEM SOLVING
Weightage:	45%	Total Marks:	25
CO2 Statement:	Apply integer and floating-point arithmetic algorithms including Booth's multiplication, restoring/non-restoring division, and IEEE 754 standard operations to solve fixed-point and floating-point computational problems. (BL3)		
Student Name:	Brasilla R	Reg.No.:	192511357
Department:	BTECH CSE	Date:	04.08.2026

ANNEXURE A

1. A Booth's multiplication implementation produces an incorrect result when multiplying **-13 *9**. Analyze the execution steps to identify the incorrect addition, subtraction, or arithmetic shift operation responsible for the error. Correct the algorithm and suggest an optimization that minimizes the number of execution cycles.
2. A processor executing the **restoring division algorithm** produces an incorrect quotient for the operation **29 /5**. Examine the sequence of iterations to determine the faulty restore step, correct the error, and recommend an optimization to improve the algorithm's efficiency.
3. During the execution of the **non-restoring division algorithm**, incorrect quotient bits are generated after the fourth iteration. Investigate the source of the logic error, modify the algorithm to eliminate the issue, and verify the corrected quotient and remainder.
4. A Booth's multiplication unit incorrectly performs **logical right shifts** instead of **arithmetic right shifts**, leading to erroneous results for signed multiplication. Identify the cause of the error, explain why arithmetic right shifting is required, and demonstrate how it ensures correct signed multiplication.
5. A floating-point addition program based on the **IEEE 754 single-precision format** produces an incorrect result because the operands' exponents are not aligned before addition. Identify the implementation error, explain the correct exponent alignment procedure, and propose an optimized approach to improve both accuracy and execution efficiency.

Code of Conduct:

I Brasilla R (192511357) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

R. Binif

MARKS ALLOCATION

	Problem Understanding (20%)	Method Selection (20%)	Solution Process (25%)	Accuracy of Calculation (20%)	Interpretation of Results (15%)
Q1					
Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation

Q1. Booth's Multiplication (-13×9)

Given

- Multiplicand (M) = **-13**
- Multiplier (Q) = **9**

Problem

The Booth's multiplication implementation gives the wrong result. We need to:

- Identify the incorrect step.
- Correct the algorithm.
- Suggest an optimization.

Step 1: Binary Representation

-13 (8-bit 2's complement)

13 = 00001101

1's = 11110010

2's = 11110011

-13 = 11110011

9 (8-bit)

00001001

Step 2: Cause of Error

The error may occur because of:

- Wrong addition of M.
- Wrong subtraction of M.
- Incorrect arithmetic right shift.

If a **logical right shift** is used instead of an **arithmetic right shift**, the sign bit becomes incorrect, giving the wrong final product.

Step 3: Correct Algorithm

For Booth's Algorithm:

- If $Q_0Q_{-1} = 01 \rightarrow A = A + M$
- If $Q_0Q_{-1} = 10 \rightarrow A = A - M$
- If $Q_0Q_{-1} = 00$ or $11 \rightarrow$ No operation
- Perform **Arithmetic Right Shift** after every iteration.

Repeat for all bits.

Step 4: Correct Result

$$-13 \times 9 = -117$$

Binary (16-bit):

11111111 10001011

Decimal:

-117

Step 5: Optimization

- Use Arithmetic Right Shift instead of Logical Right Shift.
- Skip unnecessary additions when $Q_0Q_{-1} = 00$ or 11 .
- This reduces the number of execution cycles and improves performance.

Q2. Restoring Division ($29 \div 5$)

Given

- Dividend = **29**
- Divisor = **5**

The processor produces an incorrect quotient because of a **faulty restore step**. We need to:

- Identify the error.
- Correct the algorithm.
- Suggest an optimization.

Step 1: Binary Representation

Dividend (29) = 11101_2

Divisor (5) = 00101_2

Initial values:

A = 00000

Q = 11101

M = 00101

Step 2: Restoring Division Algorithm

For each iteration:

1. Left shift (A, Q).
2. Subtract M from A.
3. If $A \geq 0$:
 - Set $Q_0 = 1$.
4. If $A < 0$:

- Restore A by adding M.
 - Set $Q_0 = 0$.
5. Repeat until all bits are processed.

Step 3: Faulty Restore Step

Error:

After subtraction, if **A becomes negative**, the processor **fails to restore** A by adding the divisor back.

Correction:

If $A < 0$:

$A = A + M$

$Q_0 = 0$

This correctly restores the partial remainder before the next iteration.

Step 4: Correct Result

$29 \div 5 = 5$

Remainder = 4

Therefore,

- **Quotient** = $00101_2 = 5$
- **Remainder** = $00100_2 = 4$

Step 5: Optimization

To improve efficiency:

- Use the **Non-Restoring Division Algorithm**, which avoids restoring A after every negative result.
- This reduces the number of addition operations and execution cycles, making the division faster.

Q3. Non-Restoring Division Algorithm

Given Problem

During the execution of the **Non-Restoring Division Algorithm**, incorrect quotient bits are generated after the **4th iteration**. We need to:

- Identify the source of the logic error.
- Correct the algorithm.
- Verify the correct quotient and remainder.

Step 1: Cause of the Error

In the **Non-Restoring Division Algorithm**, the quotient bit depends on the **sign of the accumulator (A)** after each operation.

The error occurs because:

- The wrong quotient bit is assigned after the 4th iteration, **or**
- The algorithm performs the wrong operation (addition instead of subtraction, or subtraction instead of addition).

Step 2: Correct Algorithm

For each iteration:

1. Left shift (A, Q).
2. If $A \geq 0$, perform:
 - $A = A - M$
3. If $A < 0$, perform:
 - $A = A + M$
4. Set the quotient bit:
 - If $A \geq 0$, $Q_0 = 1$
 - If $A < 0$, $Q_0 = 0$
5. Repeat for all bits.
6. After the final iteration, if $A < 0$, restore it by:
 - $A = A + M$

Step 3: Correct the Error

Incorrect logic:

- Wrong quotient bit generated after the 4th iteration.

Correction:

- Assign the quotient bit based on the sign of A.
- If the final remainder is negative, restore it by adding the divisor.

Step 4: Verified Result

Quotient = 5

Remainder = 4

Step 5: Interpretation

- The logic error was caused by incorrect quotient bit generation after the 4th iteration.
- Using the correct decision rule ($A \geq 0 \rightarrow Q_0 = 1$, $A < 0 \rightarrow Q_0 = 0$) eliminates the error.
- The final restored remainder is **4**, and the correct quotient is **5**.

Q4. Booth's Multiplication – Arithmetic Right Shift vs Logical Right Shift

Given Problem

A Booth's multiplication unit incorrectly performs **Logical Right Shifts** instead of **Arithmetic Right Shifts**, resulting in incorrect signed multiplication. We need to:

- Identify the cause of the error.
- Explain why Arithmetic Right Shift is required.
- Show how it ensures correct signed multiplication.

Step 1: Identify the Error

Error:

The multiplication unit uses a **Logical Right Shift (LRS)** instead of an **Arithmetic Right Shift (ARS)**.

In a Logical Right Shift:

- The leftmost bit is always filled with **0**.
- This changes the sign of negative numbers.

Step 2: Why Arithmetic Right Shift is Required

In Booth's multiplication, numbers are represented in **2's complement** form.

Arithmetic Right Shift (ARS):

- Preserves the **sign bit (MSB)**.
- If the number is negative, **1** is shifted into the leftmost position.
- If the number is positive, **0** is shifted into the leftmost position.

This keeps the signed value correct during each iteration.

Step 3: Example

Consider the negative number **-13**.

$-13 = 11110011$

Arithmetic Right Shift (Correct):

$11110011 \rightarrow 11111001$

(Sign bit **1** is preserved.)

Logical Right Shift (Incorrect):

$11110011 \rightarrow 01111001$

Step 4: Correct Operation

During every iteration of Booth's algorithm:

1. Perform addition or subtraction based on Booth's rules.
2. Apply **Arithmetic Right Shift** to the combined register (A, Q, Q_{-1}).
3. Repeat until all bits are processed.

This maintains the correct sign and produces the correct signed product.

Step 5: Interpretation

- **Cause of Error:** Using Logical Right Shift instead of Arithmetic Right Shift.
- **Reason:** Logical Right Shift changes the sign bit of negative numbers.
- **Solution:** Use Arithmetic Right Shift to preserve the sign bit.
- **Benefit:** Ensures correct signed multiplication results.

Q5. IEEE 754 Floating-Point Addition

Given Problem

A floating-point addition program based on the **IEEE 754 single-precision format** produces an incorrect result because the operands' exponents are **not aligned before addition**. We need to:

- Identify the implementation error.
- Explain the correct exponent alignment procedure.
- Suggest an optimized approach to improve accuracy and execution efficiency.

Step 1: Identify the Error

Error:

The program adds the mantissas (significands) **without first aligning the exponents**.

This causes incorrect floating-point results because numbers with different exponents must be expressed with the **same exponent** before addition.

Step 2: Correct Exponent Alignment Procedure

The correct IEEE 754 floating-point addition steps are:

1. Compare the exponents of both operands.
2. Shift the mantissa of the number with the **smaller exponent** to the right until both exponents are equal.
3. Add or subtract the mantissas.
4. Normalize the result if necessary.
5. Round the result according to the IEEE 754 rounding rule.
6. Store the final result in IEEE 754 single-precision format.

Step 3: Example

Suppose:

Number 1: Exponent = 130

Number 2: Exponent = 127

Difference = 3

Shift the mantissa of **Number 2** right by **3 bits** so that both exponents become **130**.

Now add the mantissas and normalize the result.

Step 4: Optimization

To improve **accuracy** and **execution efficiency**:

- Align exponents before addition.
- Use **guard, round, and sticky (GRS) bits** for accurate rounding.
- Normalize only when required to reduce unnecessary operations.
- Implement hardware support for exponent comparison and alignment to speed up execution.

Step 5: Interpretation

- **Implementation Error:** Exponents were not aligned before adding mantissas.
- **Correction:** Align exponents, then perform mantissa addition, normalization, and rounding.
- **Benefit:** Produces accurate IEEE 754 floating-point results and improves execution efficiency.